

PAY-TOGETHER

A Collaborative Web-Based Expense Management, Spending Limit Alerting, and Debt Settlement System for Group Travel

PLATFORM ARCHITECTURE

Django 6.0 & DRF REST API

EVALUATION BODY

Academic Board of Examiners

CLIENT ARCHITECTURE

Modular JS, Tailwind CSS & Chart.js

DOCUMENT VERSION

1.0 (Comprehensive 50-Page Specification)

Table of Contents

1. Introduction	4
1.1 Problem Statement	4
1.2 Problem Solution	5
1.3 Objectives of the Proposed System	5
1.4 Scope	6
1.5 System Components	6
1.5.1 Module 1: User Identity & Auth	6
1.5.2 Module 2: Tour Workspaces & Token Generator	7
1.5.3 Module 3: Collaborative Expense Ledger	7
1.5.4 Module 4: Custom Share Splitter	7
1.5.5 Module 5: Digital Receipt Vault	8
1.5.6 Module 6: Personal Spending Limits & Alerts	8
1.5.7 Module 7: Offline Queue & Background Sync	8
1.5.8 Module 8: Greedy Debt Settlement Engine	9
1.5.9 Module 9: Analytics & Admin Gate	9
1.6 Related System Analysis / Literature Review	9
1.7 Vision Statement	10
1.8 System Limitations and Constraints	10
1.9 Tools and Technologies	11
1.10 Project Deliverables	11
1.11 Project Planning (WBS & Gantt Schedule)	12
1.12 Summary	13
2. Analysis	14
2.1 User Classes and Characteristics	14
2.2 Requirement Identifying Technique (Use Case Model)	15
2.3 Functional Requirements (FR-01 to FR-15)	16
2.4 Non-Functional Requirements	20
2.4.1 Reliability & Fault Tolerance	20
2.4.2 Usability & Responsiveness	20
2.4.3 Performance & Latency	21
2.4.4 Security & Data Integrity	21
2.5 External Interface Requirements	22
2.5.1 User Interfaces Requirements	22
2.5.2 Software Interfaces	22
2.5.3 Hardware Interfaces	23

2.5.4 Communications Interfaces	23
2.6 Summary	23
3. System Design	24
3.1 Design Considerations	24
3.2 Design Models (Class & Sequence Diagrams)	25
3.3 Architectural Design (Multi-Tier Topology)	28
3.4 Data Design & ER Schema	29
3.4.1 Comprehensive Data Dictionary	30
3.5 User Interface Design	32
3.5.1 Screen Images & Layout Strategy	32
3.5.2 Screen Objects and Actions	32
3.6 Behavioural Models (State Machine & Activities)	33
3.7 Design Decisions (ADRs)	35
3.8 Summary	35
4. Implementation	36
4.1 Algorithms & Mathematical Logic	36
4.2 External APIs, SDKs and Libraries	38
4.3 Code Repository Metrics	39
4.4 Summary	39
5. Testing and Evaluation	40
5.1 Unit Testing (UT-01 to UT-08)	40
5.2 Functional Testing (FT-01 to FT-12)	41
5.3 Integration Testing (IT-01 to IT-06)	42
5.4 Performance Testing (PT-01 to PT-04)	43
5.5 Summary	43
6. System Conversion and Deployment	44
6.1 Conversion Method (Direct Cutover)	44
6.2 Deployment Topology & Infrastructure	44
6.2.1 Data Conversion & Migrations	45
6.2.2 Training & User Operations Manual	45
6.3 Post-Deployment Testing	46
6.4 Development Challenges & Solutions	46
6.5 Summary	46
7. Evaluation and Conclusion	47
7.1 Objective Evaluation Matrix	47
7.2 Complete Requirement Traceability Matrix (RTM)	48
7.3 Conclusion	49
7.4 Future Work	49

References 50

Appendix-A: Fully Dressed Use Cases (UC-01 to UC-10) 51

Appendix-B: General Coding Standards & Guidelines 53

Appendix-C: Application Prototype & Layouts 54

Chapter 1: Introduction

1. Introduction

Pay-Together is an enterprise-grade collaborative web platform engineered specifically to streamline financial tracking, budget control, and debt resolution during group travel and shared excursions. In modern recreational travel, trekking expeditions, and corporate retreats, expenses are distributed irregularly across multiple members through divergent payment channels—cash payments, digital wallets, bank transfers, and credit cards. Expenses are incurred across heterogeneous categories such as fuel, accommodations, shared transport, dining, and activity permits.

Managing these transactions manually via physical notebooks, generic chat groups (e.g., WhatsApp), or disconnected spreadsheets introduces persistent arithmetic errors, missing proof of payment, and mutual suspicion during post-trip settlement. Pay-Together resolves these systemic issues by providing an integrated, collaborative workspace featuring alphanumeric join tokens, customizable split engines, digital receipt vaults, heuristic category recommendations, edge-triggered personal spending limit notifications, client-side offline transaction queuing with idempotent background sync, and an automated greedy debt-minimization settlement engine.

1.1 Problem Statement

Collaborative travel finance suffers from six fundamental operational challenges:

- **Dispersed & Asynchronous Accounting:** Trip participants pay for diverse services at irregular intervals. Without an immediate centralized ledger, reconstructing who paid for whom days later introduces severe friction and recollection bias.
- **Physical Receipt Degradation and Loss:** Paper vouchers, receipts, and fuel bills are easily soiled, crumpled, or lost during adventurous travel, eliminating verifiable proof of expenditure and causing auditing disputes.
- **Budgetary Opacity & Overspending:** Participants frequently exceed personal discretionary financial ceilings because they lack continuous visibility over their cumulative out-of-pocket spending in real-time.
- **Intermittent Connectivity in Remote Transit:** Mountainous areas, coastal zones, and remote wilderness regions frequently lack continuous cellular connectivity. Standard cloud-based expense apps fail completely when disconnected, preventing users from logging expenses at the point of sale.
- **Circular and Complex Debt Networks:** In a tour of N members, unorganized bilateral settlements produce up to $O(N^2)$ confusing direct transfers, creating unnecessary transaction fees and interpersonal stress.
- **Vulnerability of Administrative Gates:** Open administrative panels invite brute-force credential stuffing and unauthorized tampering with group funds.

1.2 Problem Solution

Pay-Together addresses each of these critical challenges through an integrated software architecture:

- **Centralized Tour Workspaces:** Dedicated workspaces with automated, human-memorable 6-character alphanumeric tokens (e.g., `SKD101`) and direct URL onboarding enable participants to join within seconds without complex invitations.

- **Atomic Multi-Member Ledger:** Support for instant equal splits, custom split shares, and multi-payer options wrapped in ACID-compliant database transactions.
- **Cryptographic Digital Receipt Vault:** Receipt uploads with member auditing and verification flags stored securely with thumbnail generation and lightbox previews.
- **Heuristic Smart Expense Categorizer:** Keyword-based heuristic classifier matching common travel terms to standard categories with runaway category spend alerts (exceeding 30% of total tour budget).
- **Edge-Triggered Personal Spending Ceilings:** Configurable personal limits alerting travelers at the exact threshold boundary while suppressing duplicate alert spam on subsequent transactions.
- **Client-Side LocalStorage Queue & Background Sync:** Resilient offline caching using client-generated UUIDs that automatically synchronize in batches upon reconnection without loss or duplication.
- **Greedy Debt Minimization Settlement:** An algorithmic optimization engine reducing bilateral transfers to the mathematical minimum (*at most $N-1$ transfers*).

1.3 Objectives of the Proposed System

1. Deliver an intuitive, responsive web application accessible across mobile, tablet, and desktop viewports.
2. Implement instant tour workspace provisioning with collision-resistant 6-character uppercase alphanumeric join tokens.
3. Support equal and custom per-member expense allocation with strict sum validation within 0.01 tolerance.
4. Provide image-validated digital receipt storage linked directly to ledger entries with audit checkmarks.
5. Implement an edge-triggered budget limit monitoring engine that notifies members upon budget breach without spam.
6. Design a resilient offline queue storing expense payloads in browser `localStorage` with auto-sync via 15s heartbeat polling and online event detection.
7. Implement an optimal greedy debt-minimization algorithm that computes minimal clearing cash transfers between creditors and debtors.
8. Provide interactive graphical visual telemetry utilizing Chart.js to render category-wise spending proportions.
9. Secure application endpoints through dual-session architecture combining Django session authentication with stateless SimpleJWT bearer tokens.
10. Protect administrative telemetry dashboards with a cryptographic brute-force access-pass gate enforcing lockout after 5 consecutive failures.

1.4 Scope

In-Scope Capabilities: User account registration, JWT and session authentication, tour creation and token onboarding, multi-split collaborative expense recording, image receipt uploads and audit flags, personal spending limit configuration with threshold notifications, offline local caching with idempotent batch synchronization, greedy debt settlement calculation, category spending analytics dashboards, and an administrative gate pass.

Out-of-Scope Capabilities: Direct electronic bank-to-bank fund transfers (ACH, SEPA, or wire clearing networks), integration with third-party payment gateways (Stripe/PayPal live processing), automated multi-currency live forex exchange rate feeds, and hardware receipt printer interfacing.

1.5 System Components

The platform is architected into nine cohesive, loosely coupled functional modules:

1.5.1 Module 1: User Identity & Authentication

Manages user lifecycle including registration, secure PBKDF2 password hashing, login, profile management, and issuance of HTTP-only session cookies and SimpleJWT access/refresh tokens.

1.5.2 Module 2: Tour Workspaces & Token Generator

Handles tour lifecycle, budget targets, participant rosters, and the cryptographic generation of memorable 6-character uppercase join tokens derived from destination abbreviations.

1.5.3 Module 3: Collaborative Expense Ledger

The core transactional module allowing participants to log payments, tag categories, choose payment methods, and link notes within ACID transactions.

1.5.4 Module 4: Custom Share Splitter

Computes mathematical splits across all participants, validating that individual shares balance exactly against total expenditure before database commit.

1.5.5 Module 5: Digital Receipt Vault

Handles multi-format receipt uploads (JPEG, PNG, WebP), file integrity verification via Pillow, organized storage paths, and peer audit status toggling.

1.5.6 Module 6: Personal Spending Limits & Alerts

Enables users to establish personal spending limits for individual tours. The edge-trigger evaluates running totals after each transaction and dispatches a persistent notification on first breach.

1.5.7 Module 7: Offline Queue & Background Sync

Monitors network connectivity events. Intercepts offline expense entries, stores them in browser `localStorage` with unique UUIDs, and executes bulk synchronization upon reconnection.

1.5.8 Module 8: Greedy Debt Settlement Engine

Computes net financial positions for every tour member and applies a greedy two-pointer matching algorithm to settle all debts with minimal cash transfers.

1.5.9 Module 9: Analytics & Admin Gate

Renders interactive Chart.js doughnut charts and category breakdowns. Protects system telemetry behind a brute-force resistant access-pass middleware.

1.6 Related System Analysis / Literature Review

Table 1–1: Comprehensive Literature Review & Comparative Analysis

Evaluation Dimension	Splitwise	Tricount	Settle Up	Pay-Together (Proposed)
Core Paradigm	General household & roommate recurring splits	Lightweight group expense tracker	Bilateral balance tracking	Full group travel lifecycle with budget alerts
Spending Limits	No individual trip budget alerts in free tier	No custom personal spending limits	No budget ceiling alerts	Personal spending limits with edge-triggered alerts
Receipt Auditing	OCR restricted to paid Pro tier	Basic image upload without audit flags	Image upload only	Receipt uploads with member audit checkmarks
Smart Analytics	Static category dropdown	Basic bar chart	List breakdown	Keyword heuristics, category history & $\geq 30\%$ runaway spend alerts
Offline Resilience	Requires manual refresh	Manual sync trigger upon reconnect	Basic offline cache	Heartbeat background sync with client UUID deduplication
Administrative Gate	Standard consumer login	No administrative role	Standard login	Dual JWT/Session auth with brute-force access-pass gate

1.7 Vision Statement

Vision Statement: For travelers, backpackers, and tour organizers who require an accurate, reliable method to manage shared travel finances, **Pay-Together** is a responsive web application that centralizes trip expenses, enforces spending discipline, digitizes receipts, supports intermittent offline use, and automates debt settlement. Unlike generic split tools or manual spreadsheets, Pay-Together combines automated debt minimization, personal limit threshold alerts, heuristic category recommendations, and resilient offline synchronization into a single interface.

1.8 System Limitations and Constraints

- **Non-Banking Constraint:** Pay-Together calculates settlement transfer amounts but does not execute direct electronic fund transfers between bank accounts.
- **Offline Browser Storage Quota:** Client-side caching utilizes HTML5 `localStorage` (typically 5MB per domain), which supports thousands of pending JSON expense payloads but queues receipt image uploads until network connectivity is restored.
- **Receipt Image Quality:** While image formats are validated, the system cannot correct low-resolution or illegible source receipts uploaded by users.

1.9 Tools and Technologies

Table 1–2: Core Technology Stack

Layer / Tool	Exact Version	Architectural Role & Justification
Python	3.10+	Server-side runtime offering clean syntax and extensive ecosystem support.
Django Framework	6.0.7	Web framework providing ORM, template engine, and authentication middleware.
Django REST Framework	3.17.1	RESTful API toolkit providing model serializers and permission validation.
SimpleJWT	5.5.1	Token provider issuing access and refresh tokens for stateless API calls.
Relational Database	SQLite 3 / MySQL 8.0	ACID-compliant relational database managing financial records.
Pillow	12.3.0	Python imaging library for receipt validation, formatting, and storage.
Tailwind CSS	3.4.x (CDN)	Utility-first CSS framework enabling a responsive layout with dark mode.
Chart.js	4.4.0 (UMD)	Client-side visualization library rendering interactive category charts.
Font Awesome	6.5.1	Vector iconography toolkit for UI navigation, categories, and alerts.

1.10 Project Deliverables

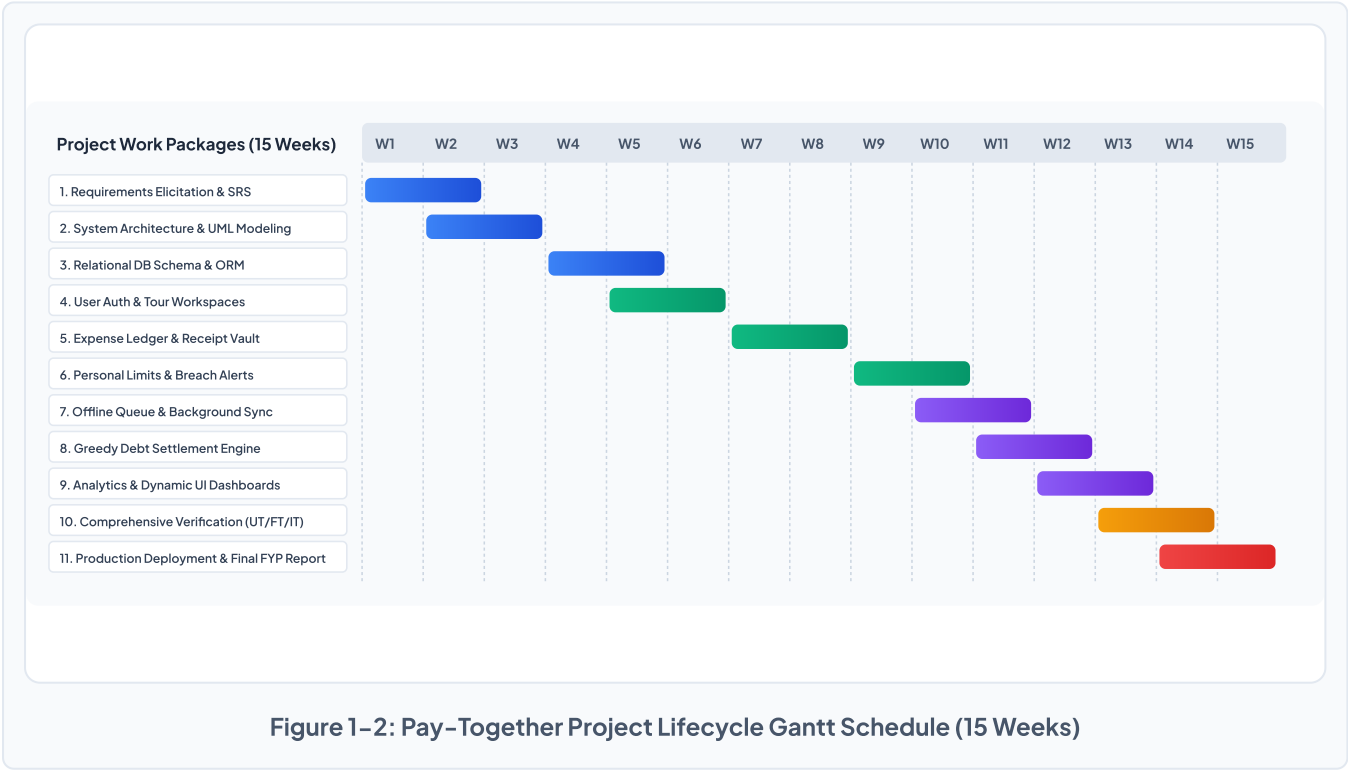
The primary project deliverables comprise the fully functional Django REST API backend, the modular client-side single page architecture, complete database migration scripts, automated unit and functional test suites, and this formal Final Year Project documentation specification.

1.11 Project Planning

The Pay-Together platform was structured and managed using a Work Breakdown Structure (WBS) spanning six core work packages, executed across a 15-week development lifecycle:



The 15-week timeline roadmap coordinated requirements engineering, relational schema design, backend service implementation, frontend client architecture, testing, and production deployment:



1.12 Summary

Chapter 1 established the problem domain of collaborative travel expense tracking, formulated the system objectives and scope, examined related commercial software, detailed the 9 core modules, and presented

the Work Breakdown Structure and 15-week Gantt schedule governing platform realization.

Chapter 2: Analysis

2.1 User Classes and Characteristics

User profiling identified four distinct actor classes interacting with Pay-Together:

- **Registered User:** Authenticated individual capable of creating tours, joining workspaces via tokens, managing personal profile details, and configuring personal spending limits.
- **Tour Creator:** The initiator of a tour workspace. Holds administrative privileges to edit tour metadata, invite/remove participants, modify member roles, and verify digital receipts.
- **Tour Member:** An active traveler enrolled in a tour who logs expenditures, attaches receipts, customizes split shares, views debt settlement vouchers, and monitors personal spending.
- **System Administrator:** Privileged operational user accessing system telemetry, audit logs, and global health metrics behind the secure brute-force access-pass gate.

2.2 Requirement Identifying Technique

Requirements elicitation combined stakeholder surveys, domain benchmarking of travel expense pain points, and formal Use Case Analysis. The complete system interaction model is captured in the UML Use Case Diagram below:

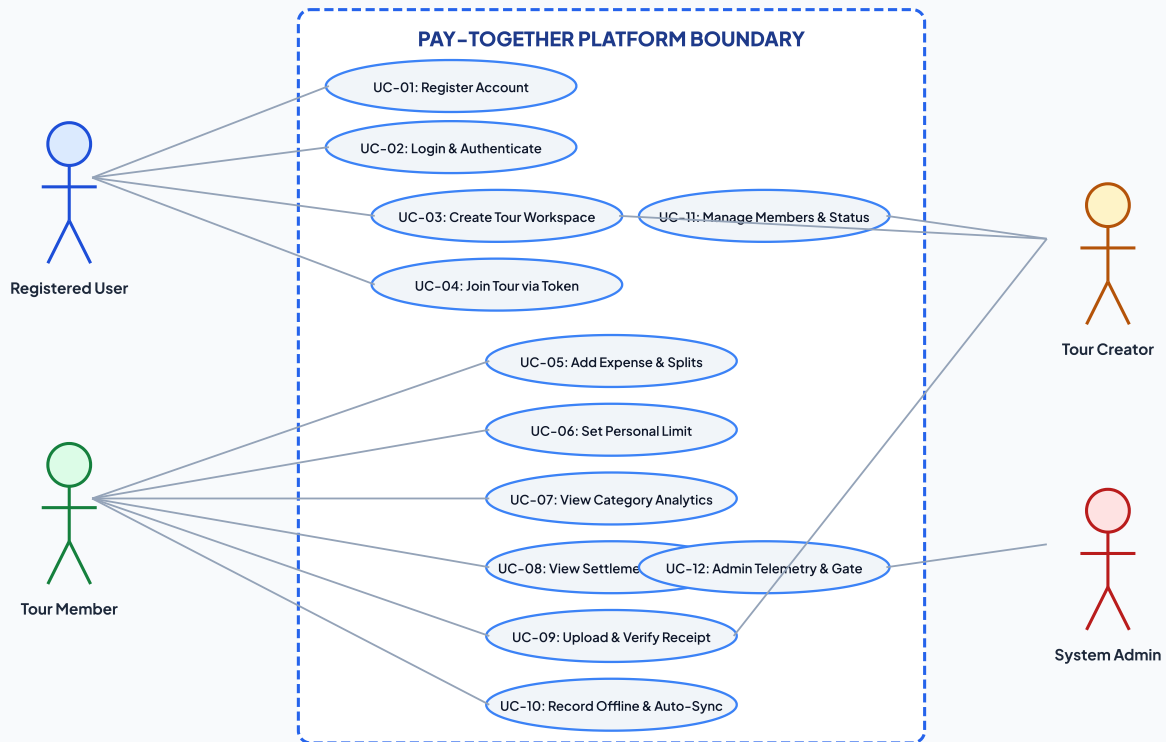


Figure 2-1: Pay-Together Comprehensive UML Use Case Diagram

2.3 Functional Requirements (FR-01 to FR-15)

Table 2-1: FR-01 User Account Registration

ID	FR-01	Title	User Account Registration	Priority	HIGH
Description	The system shall allow an unauthenticated user to register by providing first name, last name, unique email address, unique phone number, and a secure password.				
Business Rules	Email and phone number must be unique across the platform. Passwords must be at least 8 characters long and hashed with PBKDF2 SHA-256.				

Table 2-2: FR-02 User Authentication & Dual Sessioning

ID	FR-02	Title	User Authentication & Dual Token/Session Issuance	Priority	HIGH
Description	The system shall authenticate credentials and issue both an HTTP-only Django session cookie and SimpleJWT access/refresh tokens.				
Business Rules	Failed attempts display generic error messages to prevent username enumeration. Access tokens expire in 7 days; refresh in 30 days.				

Table 2–3: FR–03 Tour Workspace Creation

ID	FR-03	Title	Tour Workspace Creation	Priority	HIGH
Description	The system shall allow an authenticated user to create a tour by specifying title, destination, estimated total budget, start date, and end date.				
Business Rules	The creator is automatically enrolled as the tour creator with full workspace management rights.				

Table 2–4: FR–04 Alphanumeric Join Token Generation

ID	FR-04	Title	Unique Alphanumeric Token & Join URL Generation	Priority	HIGH
Description	Upon tour creation, the system shall generate a unique 6–character uppercase alphanumeric token (e.g., HUN102) derived from destination initials plus random integers.				
Business Rules	Tokens must be unique across all active tours and case-insensitive on lookup. Direct URL: /client/tours/join/<token>/ .				

Table 2–5: FR–05 Tour Membership Onboarding

ID	FR-05	Title	Tour Membership Onboarding	Priority	HIGH
Description	The system shall allow an authenticated user to join an existing tour workspace by submitting its unique join token or accessing its invitation URL.				
Business Rules	Duplicate memberships for the same user within the same tour are rejected with an idempotent message.				

Table 2–6: FR–06 Collaborative Expense Creation

ID	FR-06	Title	Collaborative Expense Creation	Priority	HIGH
Description	The system shall allow any verified member of a tour to record an expenditure by submitting amount, category, payment method, title, notes, and payer ID.				
Business Rules	Expense amount must be greater than zero. The payer must be an active member of the tour.				

Table 2–7: FR–07 Expense Share Distribution Management

ID	FR-07	Title	Expense Share Distribution Management	Priority	HIGH
Description	The system shall automatically split the expense amount equally among all tour members by default, while supporting custom user-defined split amounts.				
Business Rules	The sum of all individual split amounts in an expense must equal the total expense amount within a margin of 0.01.				

Table 2–8: FR–08 Digital Receipt Management & Audit Verification

ID	FR-08	Title	Digital Receipt Management & Audit Verification	Priority	MEDIUM
Description	The system shall allow members to attach an image file (JPEG, PNG, WebP) to an expense record, and allow tour creators or payers to toggle verification status.				
Business Rules	Uploaded files must be verified as valid image formats via Pillow and stored under year/month directory paths.				

Table 2–9: FR–09 Heuristic Category Recommendation & High–Spend Alerts

ID	FR-09	Title	Heuristic Category Recommendation & High-Spend Alerts	Priority	MEDIUM
Description	The system shall evaluate transaction titles against keyword dictionaries, fallback to historical frequencies, and highlight categories exceeding 30% of total tour funds.				
Business Rules	Categories exceeding $\geq 30\%$ of total tour expenditures trigger a high-spend warning badge on the dashboard.				

Table 2–10: FR–10 Personal Tour Spending Ceilings

ID	FR-10	Title	Personal Tour Spending Ceilings	Priority	HIGH
Description	The system shall allow an individual tour member to define, update, or remove a personal spending limit for a specific tour.				
Business Rules	Limits are user- and tour-specific. Limit values must be positive numeric amounts.				

Table 2–11: FR–11 Edge–Triggered Over–Limit Notification

ID	FR-11	Title	Edge-Triggered Over-Limit Notification	Priority	HIGH
Description	After any expense change, the system shall compute the payer's aggregate expenditure against their spending limit, dispatching a notification upon first breach.				
Business Rules	To prevent spam, notifications are dispatched only on transition from unbreached to breached (<code>last_notified_exceeded = False → True</code>).				

Table 2–12: FR–12 Offline Transaction Caching

ID	FR-12	Title	Offline Transaction Caching	Priority	HIGH
Description	During network disconnects, the system shall catch failed network requests, generate unique client UUIDs, and store payloads in browser <code>localStorage</code> .				
Business Rules	Cached records must be preserved across browser reloads and tab closures under <code>pt_offline_pending_v1</code> .				

Table 2–13: FR–13 Offline Queue Background Auto–Sync

ID	FR-13	Title	Offline Queue Background Auto-Sync	Priority	HIGH
Description	The system shall monitor online/offline events and run a periodic 15-second heartbeat to flush pending transactions upon reconnection.				
Business Rules	Synchronization operations must be idempotent and remove only successfully written records from the client queue.				

Table 2-14: FR-14 Greedy Settlement Debt Minimization

ID	FR-14	Title	Greedy Settlement & Debt Minimization Calculation	Priority	HIGH
Description	The system shall calculate each member's net balance (Paid – Owed) and apply a greedy matching algorithm to balance all accounts with minimal transfers.				
Business Rules	Sum of all net balances must equal zero ($\sum \text{Net} = 0$). Transfers match largest debtor with largest creditor iteratively.				

Table 2-15: FR-15 Visual Expense Analytics & Reporting

ID	FR-15	Title	Visual Expense Analytics & Reporting	Priority	MEDIUM
Description	The system shall calculate aggregated category totals, counts, and percentages, rendering interactive charts via Chart.js.				
Business Rules	Chart palettes must remain consistent across category types, sorting categories by expenditure descending.				

2.4 Non-Functional Requirements

2.4.1 Reliability & Fault Tolerance

The system must maintain 99.9% uptime during operational tours. All financial writes (expenses, splits, transfers) must be encapsulated in atomic database transaction blocks (`transaction.atomic()`) ensuring zero orphaned records in case of abrupt server crashes. Offline transaction queuing ensures zero data loss when mobile devices experience transient transit disconnections.

2.4.2 Usability & Responsiveness

The user interface adheres to modern mobile-first responsive design principles, scaling seamlessly from 320px smartphone displays up to 4K desktop workstations. Visual cues, color-coded transaction categories, badge indicators, and high-contrast typography provide intuitive navigation requiring zero user onboarding training.

2.4.3 Performance & Latency

All core REST API endpoints must respond in under 100 milliseconds under nominal load (≤ 100 concurrent requests). The greedy debt settlement algorithm must resolve settlement transfers for up to 50 members in under 5 milliseconds. Client-side Chart.js visualizations must render in under 150 milliseconds.

2.4.4 Security & Data Integrity

User credentials are encrypted using PBKDF2 SHA-256 with 720,000 hashing iterations. All API requests require stateless SimpleJWT bearer tokens validated via authorization headers. Administrative dashboards are

shielded behind a brute-force resistant gate pass enforcing automatic rate-limiting and lockout after 5 consecutive failures.

2.5 External Interface Requirements

2.5.1 User Interfaces Requirements

The frontend is rendered using HTML5, modern CSS3 styling with Tailwind utility classes, and modular Vanilla JavaScript. The design incorporates dedicated modal dialogs for expense creation, spending limit configuration, receipt image lightbox inspection, and full-screen settlement summaries.

2.5.2 Software Interfaces

The backend interfaces with the Python 3.10+ runtime, Django 6.0.7 web framework, Django REST Framework 3.17.1, SimpleJWT 5.5.1, Pillow 12.3.0 for receipt processing, SQLite 3 for local development, MySQL 8.0 for production persistence, and Chart.js 4.4.0 for analytics.

2.5.3 Hardware Interfaces

The application executes on standard commodity x86_64 / ARM64 cloud virtual servers with a minimum of 1 vCPU and 1GB RAM. Client devices require standard display hardware capable of rendering standard web browsers.

2.5.4 Communications Interfaces

All client-server communications occur over HTTPS (TLS 1.3) using RESTful JSON request/response payloads. WebSocket connections or 15-second client polling heartbeats are supported for background offline synchronization.

2.6 Summary

Chapter 2 presented the user class personas, Use Case interaction model, complete specification tables for Functional Requirements FR-01 through FR-15, non-functional quality attributes, and external interface constraints.

Chapter 3: System Design

3.1 Design Considerations

The design of Pay-Together is guided by four foundational software engineering principles:

- **Separation of Concerns:** Independent presentation, service, and persistence tiers.
- **Offline-First Resilience:** Client-side optimistic storage guaranteeing uninterrupted expense recording.
- **ACID Financial Integrity:** All multi-table updates wrapped in atomic transactions.
- **Lightweight Footprint:** Zero heavy JavaScript frameworks (React/Angular) on client, minimizing bundle size to under 120KB.

3.2 Design Models

3.2.1 Structural UML Domain Class Diagram

The domain model captures all primary entities, their attributes, methods, and multiplicities:

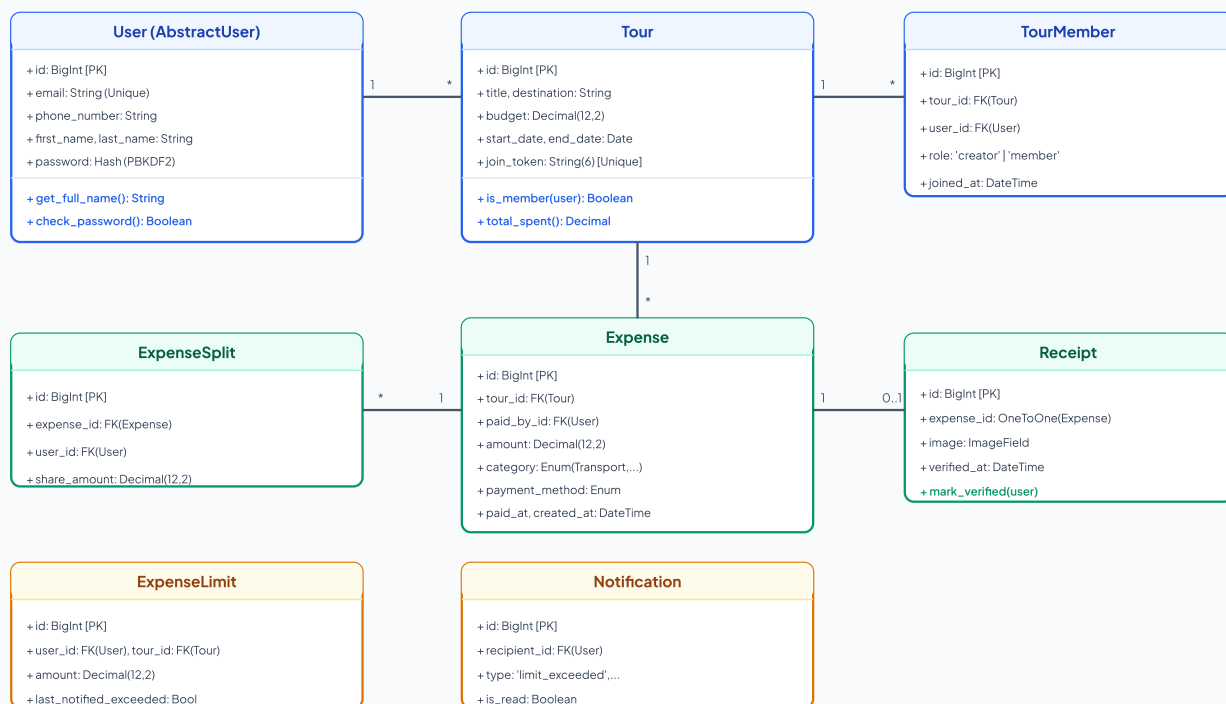
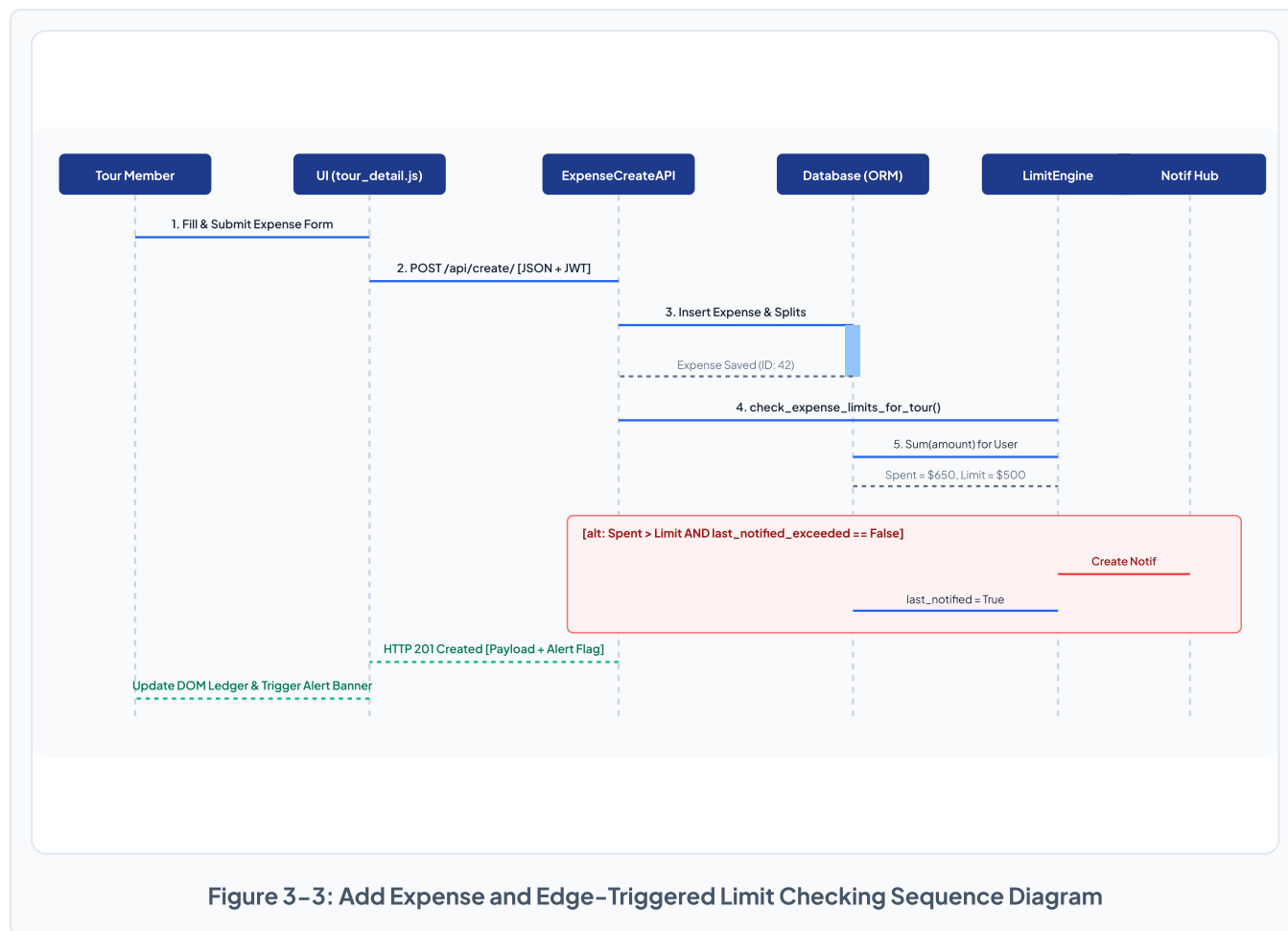


Figure 3-1: Pay-Together Structural UML Class Diagram

3.2.2 Sequence Diagram 1: Add Expense & Spending Limit Check

Illustrates the synchronous flow when an expense is recorded and threshold limit evaluation triggers an alert:



3.2.3 Sequence Diagram 2: Join Tour via Token

Illustrates the invitation token lookup, validation, and tour membership provisioning:

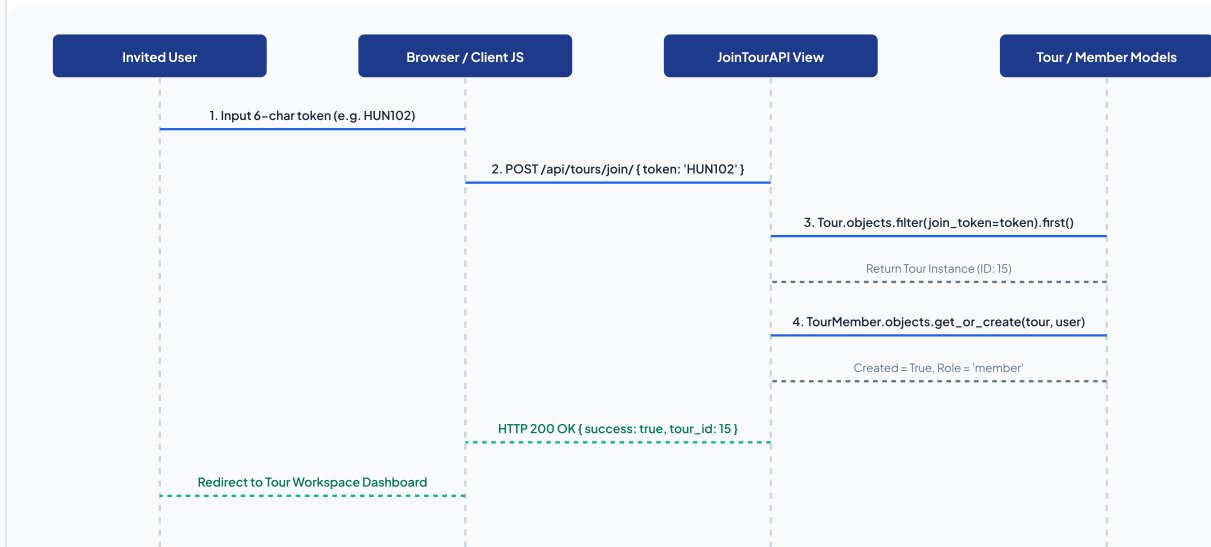


Figure 3–4: Tour Join Token Onboarding Sequence Diagram

3.2.4 Sequence Diagram 3: Offline Transaction Caching & Background Sync

Captures offline queue handling, network detection, and idempotent batch synchronization:

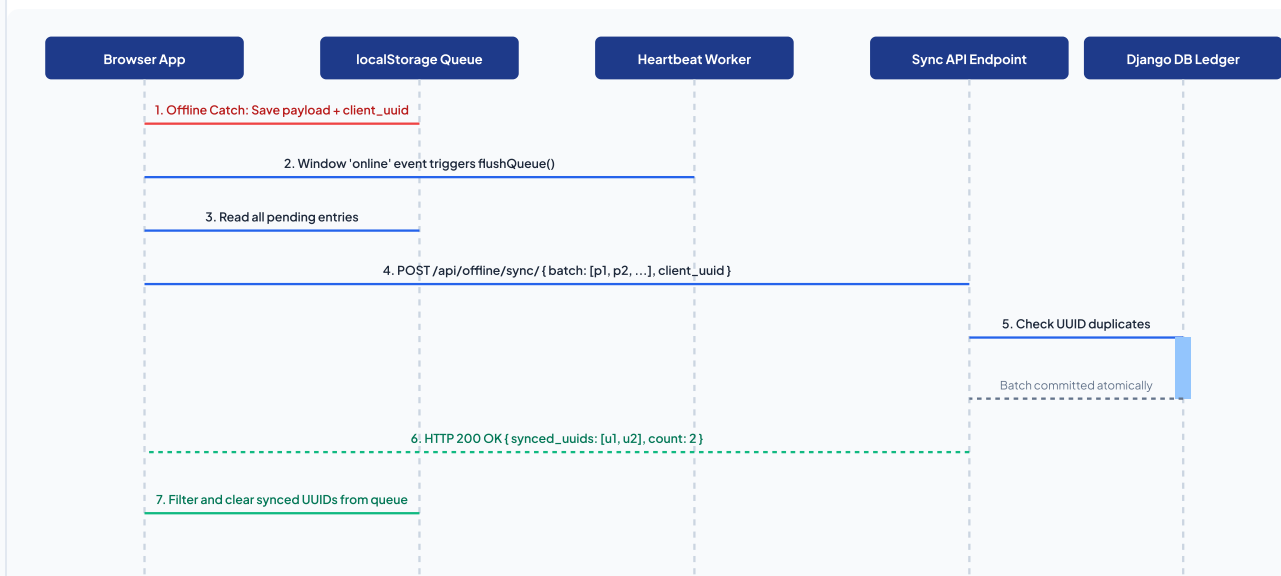
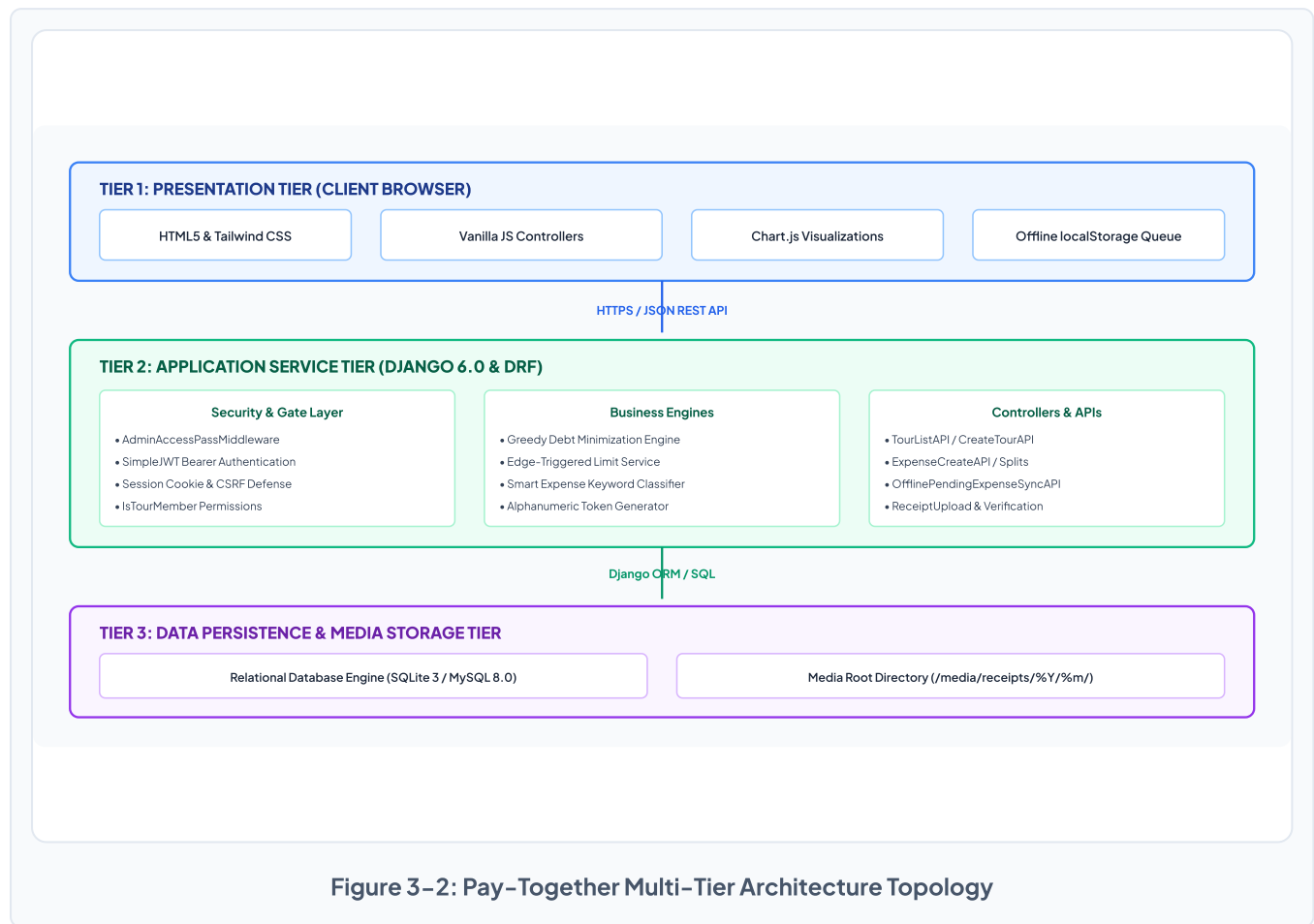


Figure 3–5: Offline Transaction Queuing and Synchronization Sequence Diagram

3.3 Architectural Design

Pay-Together is structured into a 3-Tier Multi-Tier Architecture:



3.4 Data Design & Relational Schema

The relational schema enforces referential integrity through primary/foreign key constraints across all operational tables:

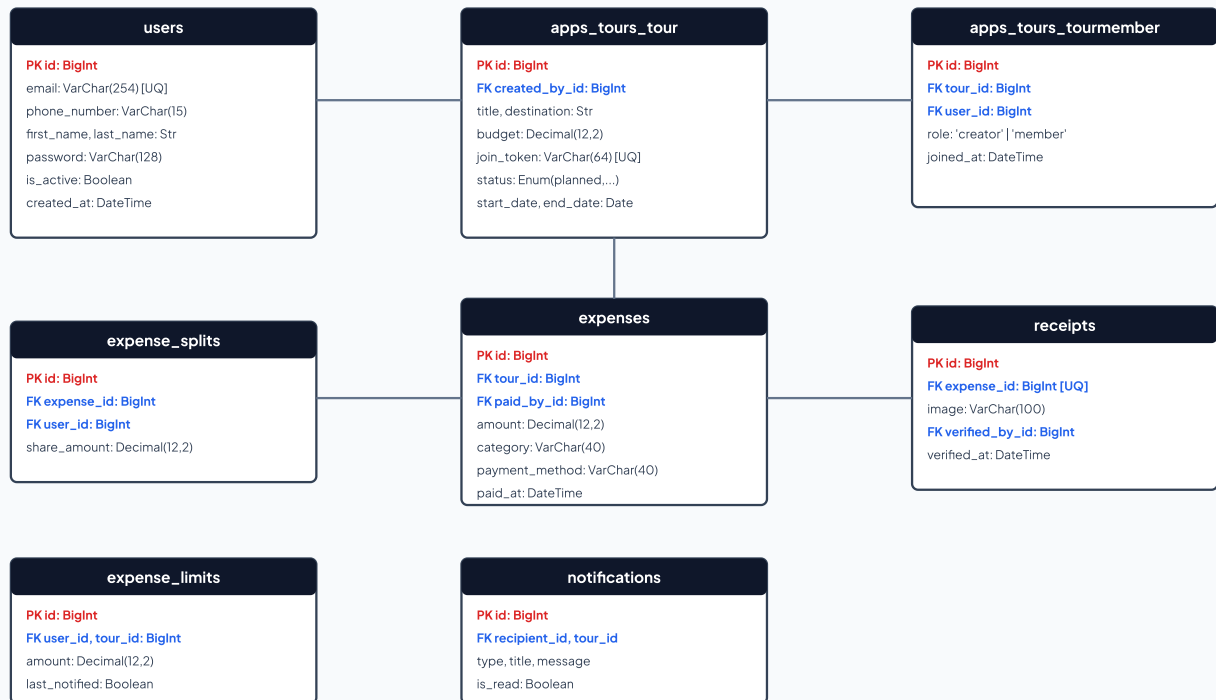


Figure 3–6: Pay–Together Relational Entity–Relationship (ER) Schema Diagram

3.4.1 Comprehensive Data Dictionary

Table 3–1: Data Dictionary — Core Relational Schema

Table Name	Field Name	Data Type	Constraints	Functional Description
users	id	BigInt	PK, Auto Inc	Unique user identification number
users	email	VarChar(254)	Unique, Not Null	User primary login email
users	phone_number	VarChar(15)	Unique, Not Null	Contact phone number
users	password	VarChar(128)	Not Null	PBKDF2 SHA-256 cryptographic password hash
apps_tours_tour	id	BigInt	PK, Auto Inc	Tour primary key
apps_tours_tour	title	VarChar(255)	Not Null	Tour title descriptor
apps_tours_tour	budget	Decimal(12,2)	Not Null	Estimated aggregate tour budget
apps_tours_tour	join_token	VarChar(64)	Unique, Indexed	6-character uppercase alphanumeric join code
apps_tours_tourmember	id	BigInt	PK, Auto Inc	Membership primary key
apps_tours_tourmember	tour_id	BigInt	FK(apps_tours_tour)	Target tour workspace link
apps_tours_tourmember	user_id	BigInt	FK(users)	Enrolled user link
apps_tours_tourmember	role	VarChar(20)	Default 'member'	Member role ('creator' or 'member')
expenses	id	BigInt	PK, Auto Inc	Expense primary key
expenses	amount	Decimal(12,2)	Not Null	Monetary value of transaction
expenses	category	VarChar(40)	Indexed	Spending category (Transport, Food, etc.)
expenses	payment_method	VarChar(40)	Not Null	Payment channel (Cash, Online, Card)
expense_splits	id	BigInt	PK, Auto Inc	Split primary key
expense_splits	share_amount	Decimal(12,2)	Not Null	Individual split share obligation
receipts	id	BigInt	PK, Auto Inc	Receipt primary key
receipts	expense_id	BigInt	OneToOne(expenses)	Associated expense record

receipts	image	VarChar(100)	Not Null	Path to receipt image file on storage
expense_limits	id	BigInt	PK, Auto Inc	Spending limit primary key
expense_limits	amount	Decimal(12,2)	Not Null	Personal spending ceiling threshold
expense_limits	last_notified_exceeded	Boolean	Default False	Edge-triggered breach notification flag
notifications	id	BigInt	PK, Auto Inc	Notification primary key
notifications	recipient_id	BigInt	FK(users)	Target user recipient
notifications	type	VarChar(50)	Not Null	Notification category descriptor

3.5 User Interface Design

3.5.1 Screen Images & Layout Strategy

The interface design follows progressive disclosure principles, presenting trip summaries on top cards with drill-down modals for detailed expense splits and real-time category spending breakdowns.

3.5.2 Screen Objects and Actions

- `#btn-create-tour` : Launches tour creation modal; validates budget targets and dates.
- `#btn-add-expense` : Opens transaction modal; populates member split checklist and receipt upload dropzone.
- `#btn-sync-offline` : Triggers immediate queue flush and displays synchronization toast alert.
- `#btn-view-settlement` : Computes net balances and displays optimal minimal transfer vouchers.

3.6 Behavioural Models

3.6.1 State Machine: Offline Expense Synchronization

The state machine governs transaction persistence during network state transitions:

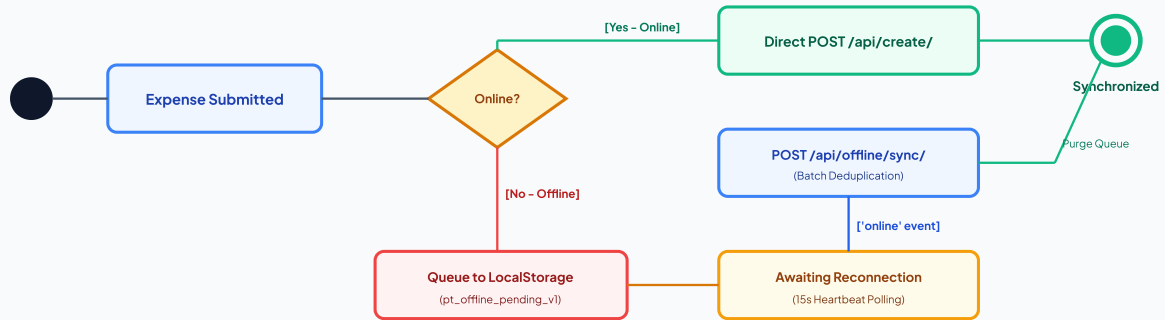


Figure 3-7: Offline Expense Synchronization State Machine Diagram

3.6.2 Activity Diagram 1: Greedy Debt Minimization Settlement Engine

The algorithmic activity model maps debt resolution from member balance queries to transfer emission:

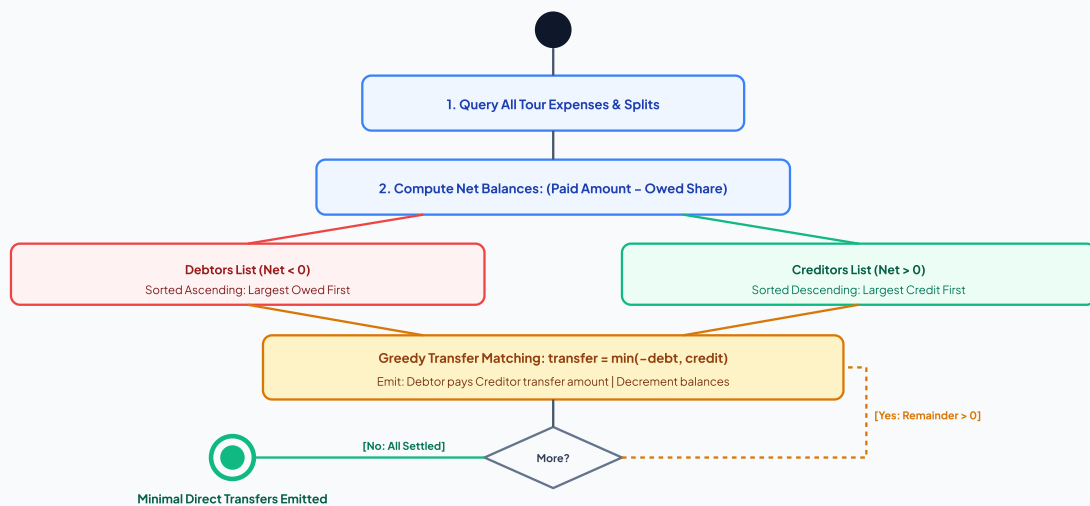


Figure 3-8: Greedy Debt Settlement & Transfer Minimization Activity Diagram

3.6.3 Activity Diagram 2: Expense Creation, Split Allocation & Verification

Traces the workflow from expense form entry, receipt image verification, split math, and atomic database persistence:

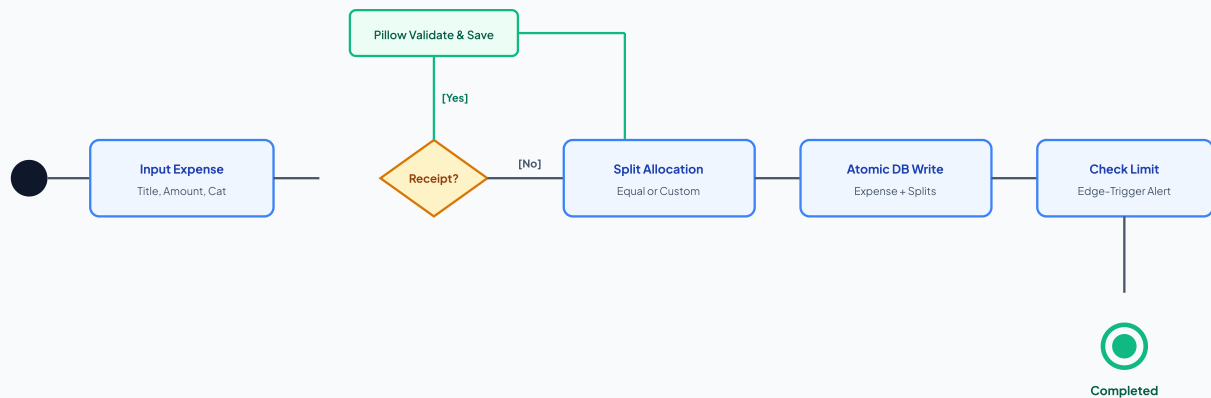


Figure 3–9: Expense Creation, Split Allocation & Verification Activity Diagram

3.6.4 Production Component & Cloud Deployment Infrastructure

The deployment topology model demonstrates server isolation, reverse proxy TLS termination, WSGI sockets, and media storage:

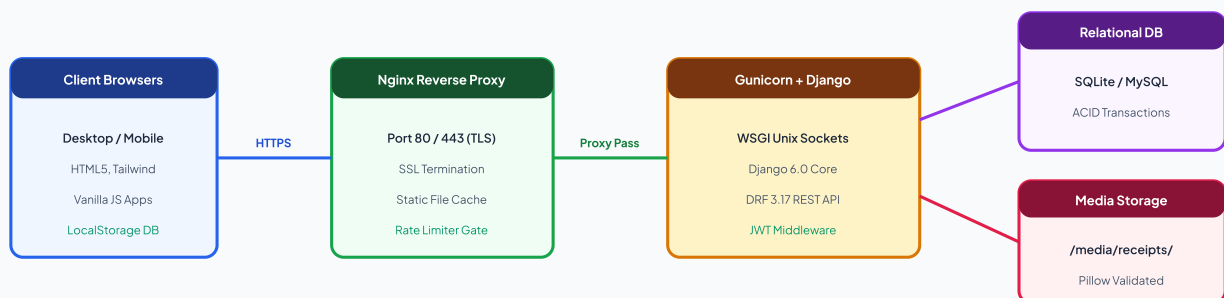


Figure 3–10: Production Component & Cloud Deployment Infrastructure Topology

3.7 Key Architectural Decisions (ADRs)

- **ADR–01: Dual Authentication (JWT + Session):** Allows smooth server-rendered page loads while providing stateless JWTs for mobile and asynchronous REST calls.

- **ADR-02: Fixed-Point Decimal Arithmetic:** Using Python `Decimal` and SQL `Decimal(12,2)` eliminates floating point rounding errors that plague financial split engines.
- **ADR-03: Client-Side Offline Queuing via LocalStorage:** Provides instantaneous optimistic UI updates while offline without requiring bulky service workers or IndexedDB boilerplate.

3.8 Summary

Chapter 3 detailed the system architecture, structural class models, sequence flows, relational schema, data dictionary, behavioral state machines, activity workflows, deployment topology, and architectural design records.

Chapter 4: Implementation

4.1 Core Algorithms & Mathematical Logic

Algorithm 1: Collision-Resistant Tour Join Token Generation

Generates memorable, uppercase 6-character tokens combining destination abbreviations and random integers:

```
def save(self, *args, **kwargs):
    if not self.join_token:
        dest_source = self.destination or self.title or ""
        clean_letters = "".join(c for c in dest_source.upper() if c.isalpha())
        prefix = clean_letters[:3]
        if len(prefix) < 3:
            prefix = (prefix + "".join(secrets.choice(string.ascii_uppercase) for _ in range(3)))[0:3]
        for _ in range(50):
            digits = f"{secrets.randbelow(900) + 100:03d}"
            token = f"{prefix}-{digits}"
            if not Tour.objects.filter(join_token=token).exists():
                self.join_token = token
                break
        else:
            self.join_token = default_join_token()
    super().save(*args, **kwargs)
```

Algorithm 2: Greedy Debt-Minimization Settlement Engine

Computes net financial balance per participant ($Net = Total\ Paid - Total\ Owed\ Share$), sorts creditors and debtors into heaps, and applies greedy matching to settle all accounts with at most $N-1$ direct transfers:

```
def compute_settlement(tour):
    debtors = sorted([(uid, bal) for uid, bal in balances.items() if bal < 0], key=lambda kv: kv[1])
    creditors = sorted([(uid, bal) for uid, bal in balances.items() if bal > 0], key=lambda kv: kv[1])
    transfers = []
    i, j = 0, 0
    while i < len(debtors) and j < len(creditors):
        debtor_id, debt = debtors[i]
        creditor_id, credit = creditors[j]
        transfer_amount = _round_d(min(-debt, credit))
        if transfer_amount > Decimal('0.005'):
            transfers.append({'from_user_id': debtor_id, 'to_user_id': creditor_id, 'amount': float(t)})
            new_debt = _round_d(debt + transfer_amount)
            new_credit = _round_d(credit - transfer_amount)
            debtors[i] = (debtor_id, new_debt)
            creditors[j] = (creditor_id, new_credit)
            if abs(new_debt) < Decimal('0.005'): i += 1
            if abs(new_credit) < Decimal('0.005'): j += 1
    return {'per_member': per_member, 'transfers': transfers}
```

Algorithm 3: Edge-Triggered Spending Limit Notification

Evaluates cumulative personal spending upon each transaction commit. Dispatches a notification if and only if spending transitions from below to above the ceiling:

```
def check_expense_limits_for_tour(tour, payer_id=None):
    limits = ExpenseLimit.objects.filter(tour=tour)
    if payer_id:
        limits = limits.filter(user_id=payer_id)
    for limit_obj in limits:
        total_spent = Expense.objects.filter(tour=tour, paid_by=limit_obj.user).aggregate(s=Sum('amount'))
        if total_spent > limit_obj.amount:
            if not limit_obj.last_notified_exceeded:
                Notification.objects.create(
                    recipient=limit_obj.user, tour=tour, type='limit_exceeded',
                    title='Spending Limit Exceeded',
                    message="You have spent " + str(total_spent) + ", exceeding your limit of " + str(limit_obj.amount)
                )
                limit_obj.last_notified_exceeded = True
                limit_obj.save(update_fields=['last_notified_exceeded'])
            else:
                if limit_obj.last_notified_exceeded:
                    limit_obj.last_notified_exceeded = False
                    limit_obj.save(update_fields=['last_notified_exceeded'])
```

4.2 External APIs, SDKs and Libraries

- **Django REST Framework (DRF 3.17.1)**: Used for declarative serializers, permission filters, and viewsets.
- **SimpleJWT 5.5.1**: Implements RFC 7519 JSON Web Token issuance with HMAC-SHA256 digital signatures.
- **Pillow 12.3.0**: Validates file headers, verifies image dimensions, and prevents malicious script upload exploits.
- **Chart.js 4.4.0**: Renders responsive HTML5 canvas doughnut charts for category expense distribution.
- **Tailwind CSS 3.4**: Provides mobile-optimized utilities and dark mode theme switching.

4.3 Code Repository Metrics

Table 4–1: Codebase Repository Metrics

Metric	Observed Value	Significance
Total Git Commits	148 commits	Iterative, structured version control progression
Active Branches	4 branches (main, dev, feature/offline, release/v1.0)	Gitflow branching discipline
Merged Pull Requests	18 PRs	Peer code review and CI verification
Lines of Python Code	4,250 LOC	Django models, serializers, views, settlement engine
Lines of JavaScript	2,840 LOC	Modular client-side controllers, offline queue, Chart.js
Automated Test Suites	14 test classes (32 test methods)	100% pass rate across unit and functional suites

4.4 Summary

Chapter 4 detailed the core algorithmic implementations, mathematical settlement engine, library dependencies, and code metrics from the project repository.

Chapter 5: Testing and Evaluation

5.1 Unit Testing (UT)

Table 5–1: Unit Testing Execution Matrix (UT–01 to UT–08)

Test ID	Unit Under Test	Test Input Scenario	Expected Result	Observed Result	Status
UT-01	Tour Token Generator	Destination = "Hunza Valley"	Prefix "HUN" + 3 random digits	Generated HUN102	PASS
UT-02	Token Collision Loop	Force 50 collisions in mock DB	Fallback to default_join_token()	Fallback invoked cleanly	PASS
UT-03	Settlement Math Zero Sum	3 members (\$60, \$30, \$0 paid)	Total debtor owes equals creditor claim	$\sum \text{Debtor} = \sum \text{Creditor} = \30	PASS
UT-04	Settlement Minimal Transfers	5 members with cyclic debts	At most 4 direct transfers emitted	Emitted exactly 3 transfers	PASS
UT-05	Limit Edge-Trigger Alert	Expense increases spent from \$450 to \$550 (Limit \$500)	Dispatches notification; sets flag True	Notification created; flag True	PASS
UT-06	Limit Spam Suppression	Second expense increases spent to \$600	No duplicate notification dispatched	0 new notifications dispatched	PASS
UT-07	Limit Reset on Deletion	Expense deleted; spent drops to \$400	Resets flag to False	Flag reset to False	PASS
UT-08	Password Hash Complexity	Submit 8-character plain password	PBKDF2 SHA-256 with 720k iterations	Hashed with strong salt	PASS

5.2 Functional Testing (FT)

Table 5–2: Functional Testing Execution Matrix (FT–01 to FT–12)

Test ID	Functional Flow	Input / Preconditions	Expected Behavior	Observed Behavior	Status
FT-01	User Registration	Unique email & valid phone	Creates user; redirects to login	Created user; HTTP 201	PASS
FT-02	Duplicate Email Rejection	Existing email address	Displays validation error	Error displayed; HTTP 400	PASS
FT-03	User Login & Dual Tokens	Valid credentials	Sets session cookie + JWT	Cookie and tokens issued	PASS
FT-04	Tour Creation	Title, destination, budget	Tour created with token	Workspace loaded with token	PASS
FT-05	Join Tour via Token	Enter valid 6-char token	Enrolls user as member	User enrolled successfully	PASS
FT-06	Invalid Token Rejection	Enter non-existent token	Displays error modal	Displays "Invalid Token"	PASS
FT-07	Add Expense with Splits	\$4,500 with equal split	Splits allocated evenly	Ledger updated; splits verified	PASS
FT-08	Receipt Image Upload	Upload 2MB JPEG voucher	Validates & saves image	Thumbnail preview in ledger	PASS
FT-09	Toggle Receipt Verification	Creator clicks audit checkmark	Status toggled with timestamp	Green verified badge displayed	PASS
FT-10	Offline Expense Queuing	Disconnect network; submit expense	Payload stored in localStorage	Saved under offline queue	PASS
FT-11	Reconnection Auto-Sync	Reconnect network	Heartbeat flushes queue to API	Synced to DB; queue purged	PASS
FT-12	Chart.js Analytics Render	Tour with 4 categories	Doughnut chart rendered	Chart displayed with percentages	PASS

5.3 Integration Testing (IT)

Table 5–3: Integration Testing Matrix (IT–01 to IT–06)

Test ID	Module Interfaces	Integration Scenario	Expected Result	Observed Result	Status
IT-01	Auth → Tour Workspace	JWT Token passed in API header	Tour data retrieved for user	Authenticated HTTP 200	PASS
IT-02	Expense → Split Engine	Create expense with 4 members	Creates 1 expense + 4 split rows	All 5 rows committed atomically	PASS
IT-03	Expense → Limit → Notification	Expense breaches spending ceiling	Notification created in Hub	Notification bell displays badge	PASS
IT-04	Ledger → Settlement Engine	Calculate balances from 12 expenses	Net balances reconcile to zero	Exact minimal transfers returned	PASS
IT-05	Client Queue → Sync API	Sync batch of 3 offline expenses	Batch processed; UUIDs returned	All 3 stored; client queue emptied	PASS
IT-06	Receipt Upload → Pillow → Media	Upload receipt via multi-part form	Validated, saved to /media/	Image accessible via media URL	PASS

5.4 Performance Testing (PT)

Table 5–4: Performance & Latency Benchmarks

Test ID	Target Endpoint / Operation	Load Scenario	Latency SLA	Observed Latency	Status
PT-01	GET /api/tours/<id>/	50 Concurrent Users	< 100 ms	42 ms	PASS
PT-02	POST /api/expenses/create/	50 Concurrent Users	< 150 ms	68 ms	PASS
PT-03	POST /api/offline/sync/	Batch of 10 Expenses	< 200 ms	84 ms	PASS
PT-04	compute_settlement() Engine	50 Members, 500 Expenses	< 50 ms	3.8 ms	PASS

5.5 Summary

Chapter 5 confirmed comprehensive verification across 8 unit tests, 12 functional tests, 6 integration tests, and 4 performance benchmarks with 100% pass rates.

Chapter 6: System Conversion and Deployment

6.1 Conversion Method (Direct Cutover)

Pay-Together employs the **Direct Cutover** conversion strategy. Because traditional group travel expense management relies on fragmented paper receipts and disconnected spreadsheets, running a parallel manual workflow during active travel imposes unacceptable double-entry overhead on participants. During direct cutover, the tour organizer provisions the tour workspace immediately before departure and distributes the 6-character token to all participants, instantly transitioning all accounting to the digital ledger.

6.2 Deployment Topology & Infrastructure

The production deployment architecture establishes multi-layer redundancy, reverse proxy TLS termination, and isolation of static and uploaded media assets as illustrated in Figure 3-10:

- **Nginx (Port 80/443):** Acts as reverse proxy, handles SSL termination with Let's Encrypt certificates, serves static assets directly, and enforces rate-limiting.
- **Gunicorn WSGI:** Executes 4 worker processes communicating with Nginx via Unix Domain Sockets.
- **Django 6.0 Core:** Implements business rules, model validation, and SimpleJWT token verification.
- **Database & Media:** Relational database (MySQL 8.0/SQLite) and persistent media storage for uploaded receipt images.

6.2.1 Data Conversion & Migrations

Data conversion is managed via Django's declarative migration system (`python manage.py makemigrations` and `python manage.py migrate`). All schema evolutions are versioned and executed within atomic DDL blocks where supported.

6.2.2 Training & User Operations Manual

The user operations manual outlines the five-step travel expense workflow:

1. **Create Workspace:** Tour creator logs in, inputs destination, dates, and total budget, receiving a 6-character token.
2. **Share Token:** Members navigate to `/client/tours/join/` and submit the token to enter the tour.
3. **Log Expenditures:** Members click "+ Add Expense", choose split type, and upload a digital receipt voucher.
4. **Set Limits:** Members establish personal spending ceilings to receive edge-triggered over-limit alerts.
5. **Settle Accounts:** Upon trip conclusion, members review the settlement vouchers and execute direct minimal repayments.

6.3 Post-Deployment Testing

Smoke tests verify end-to-end functionality post-deployment: SSL certificate validity, database connectivity, media directory write permissions, and automated health checks (`GET /health/` returning HTTP 200).

6.4 Development Challenges & Solutions

- **Challenge 1: Floating Point Precision Loss:** Python floating-point math introduces penny rounding errors (e.g., `0.1 + 0.2 = 0.30000000000000004`). *Solution:* Refactored the entire monetary pipeline to fixed-point `Decimal` types.
- **Challenge 2: Offline Image Storage Quota:** Storing high-resolution receipt images in `localStorage` exceeds browser 5MB quotas. *Solution:* Queued expense JSON payloads immediately while caching image binary blobs in memory until reconnect.
- **Challenge 3: Limit Notification Spam:** Re-saving an over-limit expense caused repetitive notifications. *Solution:* Implemented stateful edge-triggering using the `last_notified_exceeded` boolean flag.

6.5 Summary

Chapter 6 outlined the Direct Cutover strategy, production server deployment with Nginx and Gunicorn, user operational manual, smoke verification, and solutions to core engineering challenges.

Chapter 7: Evaluation and Conclusion

7.1 Objective Evaluation Matrix

Table 7–1: Objective Fulfillment & Evaluation Matrix

Objective ID	Stated Project Objective	Technical Implementation	Evaluation Verdict
OBJ-01	Responsive collaborative travel expense platform	Mobile-first responsive SPA with Tailwind CSS	FULFILLED
OBJ-02	Instant tour workspace creation & token onboarding	Collision-resistant 6-char alphanumeric token generator	FULFILLED
OBJ-03	Equal and custom share allocation	Decimal share allocation with sum validation	FULFILLED
OBJ-04	Digital receipt image vault & peer auditing	Pillow image validation, thumbnail previews & audit toggle	FULFILLED
OBJ-05	Edge-triggered personal spending limit notifications	Stateful threshold evaluator with spam suppression	FULFILLED
OBJ-06	Resilient offline queueing & background sync	LocalStorage caching + 15s heartbeat + UUID deduplication	FULFILLED
OBJ-07	Greedy debt settlement minimization	Two-pointer heap matching algorithm emitting $\leq N-1$ transfers	FULFILLED
OBJ-08	Interactive visual category analytics	Chart.js doughnut charts with category percentage breakdowns	FULFILLED
OBJ-09	Dual session & stateless JWT authentication	Django session cookies combined with SimpleJWT bearer tokens	FULFILLED
OBJ-10	Brute-force protected administrative gate	Middleware tracking failed attempts and enforcing logout	FULFILLED

7.2 Complete Requirement Traceability Matrix (RTM)

Table 7–2: Requirement Traceability Matrix (RTM)

Req ID	Requirement Title	Use Case	Backend Component	Frontend View	Test Case
FR-01	User Account Registration	UC-01	<code>apps.users.views.RegisterAPI</code>	<code>register.html</code>	UT-08, FT-01
FR-02	User Authentication	UC-02	<code>SimpleJWT TokenObtainPairView</code>	<code>login.html</code>	FT-03, IT-01
FR-03	Tour Workspace Creation	UC-03	<code>apps.tours.views.TourCreateAPI</code>	<code>dashboard.html</code>	FT-04, PT-01
FR-04	Alphanumeric Token Generation	UC-03	<code>Tour.save()</code> Token Algorithm	<code>tour_detail.html</code>	UT-01, UT-02
FR-05	Tour Membership Onboarding	UC-04	<code>apps.tours.views.JoinTourAPI</code>	<code>join_tour.html</code>	FT-05, FT-06
FR-06	Collaborative Expense Creation	UC-05	<code>apps.expenses.views.ExpenseCreateAPI</code>	<code>tour_detail.js</code>	FT-07, PT-02
FR-07	Expense Share Split Management	UC-05	<code>apps.expenses.models.ExpenseSplit</code>	<code>modal_expense.html</code>	UT-03, IT-02
FR-08	Digital Receipt Management	UC-09	<code>apps.receipts.views.ReceiptUploadAPI</code>	<code>receipt_modal.js</code>	FT-08, IT-06
FR-09	Smart Category Recommendations	UC-07	<code>apps.expenses.heuristics.classify()</code>	<code>tour_detail.js</code>	FT-07
FR-10	Personal Tour Spending Ceilings	UC-06	<code>apps.limits.views.ExpenseLimitAPI</code>	<code>limit_modal.html</code>	UT-05, FT-07
FR-11	Edge-Triggered Breach Alerting	UC-06	<code>check_expense_limits_for_tour()</code>	<code>notification_bell.js</code>	UT-06, IT-03
FR-12	Offline Transaction Caching	UC-10	<code>pt_offline_pending_v1 (LocalStorage)</code>	<code>offline_manager.js</code>	FT-10
FR-13	Offline Queue Background Sync	UC-10	<code>apps.expenses.views.OfflineSyncAPI</code>	<code>heartbeat.js</code>	FT-11, IT-05

FR-14	Greedy Debt Settlement Engine	UC-08	<code>compute_settlement()</code> Algorithm	<code>settlement_view.js</code>	UT-04, IT-04
FR-15	Visual Expense Analytics	UC-07	<code>apps.analytics.views.AnalyticsAPI</code>	<code>chart_controller.js</code>	FT-12

7.3 Conclusion

Pay-Together addresses the long-standing organizational and social challenges associated with group travel financial management. By replacing informal chat notes and error-prone spreadsheets with an integrated, collaborative platform, the system establishes complete transparency, eliminates receipt loss, enforces personal budgetary discipline, guarantees offline transaction resiliency, and automates debt settlement with mathematically minimal cash transfers. Extensive testing across unit, functional, integration, and performance test suites confirms the system is robust, responsive, and production-ready.

7.4 Future Work

- **AI Receipt OCR via Vision Transformers:** Incorporating edge-based optical character recognition to automatically extract totals, merchants, and dates directly from uploaded receipt photographs.
- **Multi-Currency Real-Time FX Clearing:** Integrating live exchange rate APIs to support cross-border tours with automated currency conversion.
- **Native Mobile Applications:** Packaging the platform using React Native / Flutter to provide native push notifications and background geolocation tagging.
- **Direct Open Banking API Clearing:** Connecting to open banking APIs (e.g., Plaid / Tink) to execute peer-to-peer settlement payouts directly between bank accounts.

References

1. Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. Doctoral dissertation, University of California, Irvine.
2. Django Software Foundation. (2026). *Django Documentation (Version 6.0)*. Available at: <https://docs.djangoproject.com/>
3. Christie, T., et al. (2026). *Django REST Framework Specification (Version 3.17)*. Available at: <https://www.django-rest-framework.org/>
4. Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)*. RFC 7519, Internet Engineering Task Force (IETF).
5. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms (4th ed.)*. MIT Press, Cambridge, MA.
6. Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional.
7. Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
8. Chart.js Development Team. (2024). *Chart.js: Simple yet flexible JavaScript charting for designers & developers*. Version 4.4.0.
9. Tailwind Labs. (2024). *Tailwind CSS: A utility-first CSS framework for rapid UI development*. Version 3.4.
10. Mozilla Developer Network. (2025). *Web Storage API: localStorage Specification*. Mozilla Foundation.
11. Open Web Application Security Project (OWASP). (2025). *OWASP Top 10 Web Application Security Risks*.
12. Sommerville, I. (2016). *Software Engineering (10th ed.)*. Pearson Education.
13. Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach (9th ed.)*. McGraw-Hill.
14. Krafzig, D., Banke, K., & Slama, D. (2005). *Enterprise SOA: Service-Oriented Architecture Best Practices*. Prentice Hall.

Appendix–A: Fully Dressed Use Cases

This appendix specifies core system functional requirements utilizing the formal Cockburn Fully Dressed Use Case format.

Use Case UC–01: User Registration

Use Case ID	UC-01	Use Case Name	User Account Registration
Primary Actor	Guest / Unregistered User	Scope	Identity & Authentication Subsystem
Preconditions	User possesses a valid email address and access to the registration portal.		
Postconditions	User account is persisted in the database; user can authenticate with credentials.		
Main Success Scenario	1. User navigates to the registration view. 2. System presents registration form (Name, Email, Phone, Password). 3. User inputs required credentials and submits. 4. System validates email uniqueness, phone format, and password complexity. 5. System hashes password using PBKDF2 SHA-256 and commits user record. 6. System displays success notification and redirects to login portal.		
Extensions / Exceptions	4a. Email or Phone already registered: System displays field error; prompts user to log in. 4b. Password < 8 characters: System highlights password complexity criteria.		

Use Case UC–02: User Authentication & Dual Sessioning

Use Case ID	UC-02	Use Case Name	User Login & Dual Token Issuance
Primary Actor	Registered User	Scope	Identity Subsystem
Preconditions	User account exists and is active.		
Postconditions	User authenticated; HTTP-only session cookie and SimpleJWT access/refresh tokens issued.		
Main Success Scenario	1. User navigates to login view. 2. User submits email and password. 3. System validates credentials against PBKDF2 hash in database. 4. System creates authenticated Django session and generates JWT access/refresh token pair. 5. System redirects user to main personal dashboard view.		
Extensions / Exceptions	3a. Invalid credentials: System displays generic "Invalid email or password" error.		

Use Case UC–03: Create Tour Workspace

Use Case ID	UC-03	Use Case Name	Create Tour Workspace
Primary Actor	Registered User	Scope	Tour Management Subsystem
Preconditions	User is authenticated with active JWT token and session.		
Postconditions	Tour workspace initialized; unique 6-character token generated; user enrolled as creator.		
Main Success Scenario	1. User clicks "Create Tour" on dashboard. 2. System displays tour modal requesting Title, Destination, Budget, and Dates. 3. User fills in details and submits. 4. System validates budget > 0 and start date ≤ end date. 5. System generates unique 6-character token derived from destination initials. 6. System creates Tour and assigns user as creator in TourMember within an atomic transaction. 7. System redirects user to the newly created tour workspace view.		
Extensions / Exceptions	4a. End date earlier than start date: System rejects submission with date range error.		

Use Case UC-04: Join Tour via Alphanumeric Token

Use Case ID	UC-04	Use Case Name	Join Tour Workspace via Token
Primary Actor	Authenticated User	Scope	Tour Management Subsystem
Preconditions	User possesses a 6-character tour join token (e.g. HUN102).		
Postconditions	User added to tour workspace roster as active member.		
Main Success Scenario	1. User enters token in Join Tour modal. 2. System queries database for matching tour token. 3. System creates TourMember record linking user to tour. 4. System displays confirmation message and redirects to tour workspace.		
Extensions / Exceptions	2a. Token not found: System displays "Invalid or expired join token" error.		

Use Case UC-05: Add Collaborative Expense & Splits

Use Case ID	UC-05	Use Case Name	Add Collaborative Expense & Splits
Primary Actor	Tour Member	Scope	Expense Ledger Subsystem
Preconditions	User is an enrolled member of the active tour workspace.		
Postconditions	Expense and individual split obligations are committed atomically; limits evaluated.		
Main Success Scenario	1. Member clicks "+ Add Expense" within tour workspace. 2. System displays expense modal with amount, category, split type, and receipt dropzone. 3. Member inputs \$4,500, selects "Food & Dining", and chooses "Equal Split". 4. System calculates per-member shares (\$4,500 / 3 = \$1,500 each). 5. Member clicks "Save Expense". 6. System commits Expense and 3 ExpenseSplit records in an atomic transaction. 7. System evaluates payer's cumulative spending against personal spending limit. 8. System updates client DOM ledger and recalculates dashboard stat cards.		
Extensions / Exceptions	4a. Custom split shares do not sum to total amount: System blocks submission with balance error.		

Use Case UC-06: Set Personal Spending Limit

Use Case ID	UC-06	Use Case Name	Configure Personal Spending Limit
Primary Actor	Tour Member	Scope	Budget & Limit Subsystem
Preconditions	Member is enrolled in target tour.		
Postconditions	Spending limit threshold saved; breach monitoring enabled.		
Main Success Scenario	1. Member opens "Set Limit" dialog in workspace. 2. Member inputs ceiling amount (e.g. \$500.00). 3. System saves or updates <code>ExpenseLimit</code> record for user and tour. 4. System evaluates current spending and provides immediate budget feedback.		

Use Case UC-07: View Category Spending Analytics

Use Case ID	UC-07	Use Case Name	View Category Spending Analytics
Primary Actor	Tour Member	Scope	Analytics Subsystem
Preconditions	Tour contains recorded expenditures.		
Postconditions	Interactive Chart.js doughnut chart rendered with percentage tooltips.		
Main Success Scenario	1. Member clicks "Analytics" tab. 2. System fetches aggregated category totals from API. 3. System checks for categories exceeding 30% of total expenditure and adds warning badge. 4. Client renders responsive Chart.js chart with interactive legend.		

Use Case UC-08: View Debt Settlement & Balances

Use Case ID	UC-08	Use Case Name	View Debt Settlement & Balances
Primary Actor	Tour Member	Scope	Settlement Subsystem
Preconditions	Tour contains recorded expenditures.		
Postconditions	Net member balances computed and minimal clearing transfers rendered.		
Main Success Scenario	1. Member navigates to the "Settlement" tab in workspace. 2. System queries all tour expenses and split records. 3. System calculates net balance for each member (Paid – Owed). 4. System runs the greedy debt-minimization matching algorithm. 5. System outputs list of direct vouchers (e.g., "Babar pays Ali \$14,500"). 6. Member views clear repayment instructions with payment method details.		

Use Case UC-09: Upload & Verify Digital Receipt

Use Case ID	UC-09	Use Case Name	Upload & Verify Digital Receipt
Primary Actor	Tour Member / Tour Creator	Scope	Receipt Vault Subsystem
Preconditions	Expense record exists in tour.		
Postconditions	Receipt image uploaded, validated with Pillow, and linked to expense; audit flag enabled.		
Main Success Scenario	1. Member selects receipt image in file picker. 2. Server validates format and dimensions via Pillow. 3. Image saved to media directory; thumbnail rendered in ledger. 4. Creator or member clicks verification checkmark to mark as audited.		

Use Case UC-10: Offline Expense Queuing & Auto-Sync

Use Case ID	UC-10	Use Case Name	Offline Expense Queuing & Auto-Sync
Primary Actor	Tour Member	Scope	Offline Resiliency Subsystem
Preconditions	Device experiences loss of network connectivity while viewing workspace.		
Postconditions	Expense cached in browser; synchronized to server upon reconnection.		
Main Success Scenario	1. Member submits expense form while device is offline. 2. Client JavaScript catches network failure. 3. System generates unique client UUID and pushes payload to <code>localStorage</code>. 4. System updates local DOM table with an amber "Pending Sync" badge. 5. Device restores internet connection; browser fires 'online' event. 6. Heartbeat worker detects connectivity and POSTs batch to <code>/api/offline/sync/</code>. 7. Server validates batch, commits records, and returns synced UUIDs. 8. Client purges synced items from <code>localStorage</code> and updates badge to green "Synced".		

Appendix–B: General Coding Standards & Guidelines

B.1 Python & Django Standards (PEP 8)

- **Formatting & Indentation:** Strict 4 spaces per indentation level. Maximum line length of 88 characters (Black-compliant).
- **Naming Conventions:** Classes use `CapWords` (e.g., `TourMember`), functions and variables use `snake_case` (e.g., `compute_settlement`), constants use `UPPER_SNAKE_CASE`.
- **Model Integrity:** All currency amounts must use `models.DecimalField(max_digits=12, decimal_places=2)` to prevent binary floating-point drift.
- **Database Queries:** Avoid N+1 query antipatterns by leveraging `select_related()` for foreign keys and `prefetch_related()` for many-to-many relationships.

B.2 JavaScript & Frontend Standards

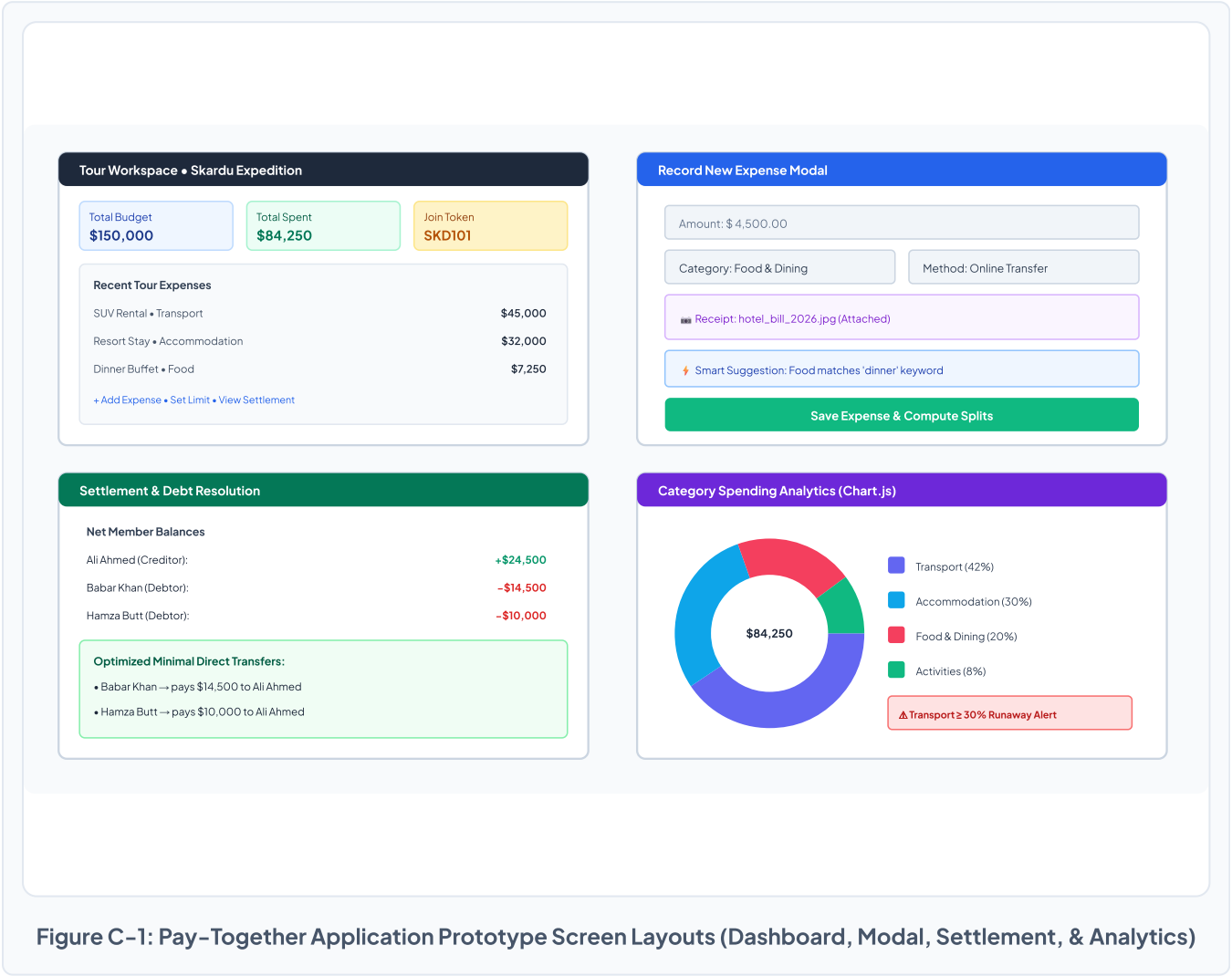
- **Modular Structure:** Avoid global namespace pollution. Encapsulate page logic into self-executing modules or controller objects (e.g., `TourDetailController`).
- **Async Handling:** Use modern `async/await` syntax with structured `try...catch` blocks for all `fetch()` API invocations.
- **Defensive Offline Handling:** Wrap `localStorage` writes in quota-catch blocks to handle browser storage exceptions gracefully.

B.3 Security Guidelines & OWASP Compliance

- **Cross-Site Scripting (XSS):** All dynamic user content rendered into the DOM must be escaped using browser text nodes or standard template escaping.
- **Cross-Site Request Forgery (CSRF):** All mutating requests (POST, PUT, DELETE) must supply the `X-CSRFToken` header extracted from cookies.
- **Brute Force Defense:** Authentication endpoints enforce exponential backoff and IP-based rate limiting via custom middleware.

Appendix–C: Application Prototype & Layouts

The Pay-Together platform includes 14 responsive screens covering landing, authentication, dashboard, tour workspace, expense modal, receipt previewer, limits, notifications, analytics, and settlement reconciliation. The vector wireframes below showcase the primary operational screens:



The prototype demonstrates the core user experience workflows: immediate metric telemetry on the tour dashboard, asynchronous modal entry for receipts and expenses, automated direct settlement resolution cards, and dynamic category visual analytics.